Embedded System HW Design
Week3
Selecting Microcontrollers

- Selecting Microcontrollers
- Compare the different type of processor architectures
- ARM Cortex-M architecture

Template designed by Tim Palmer tpalmer@vsu.edu

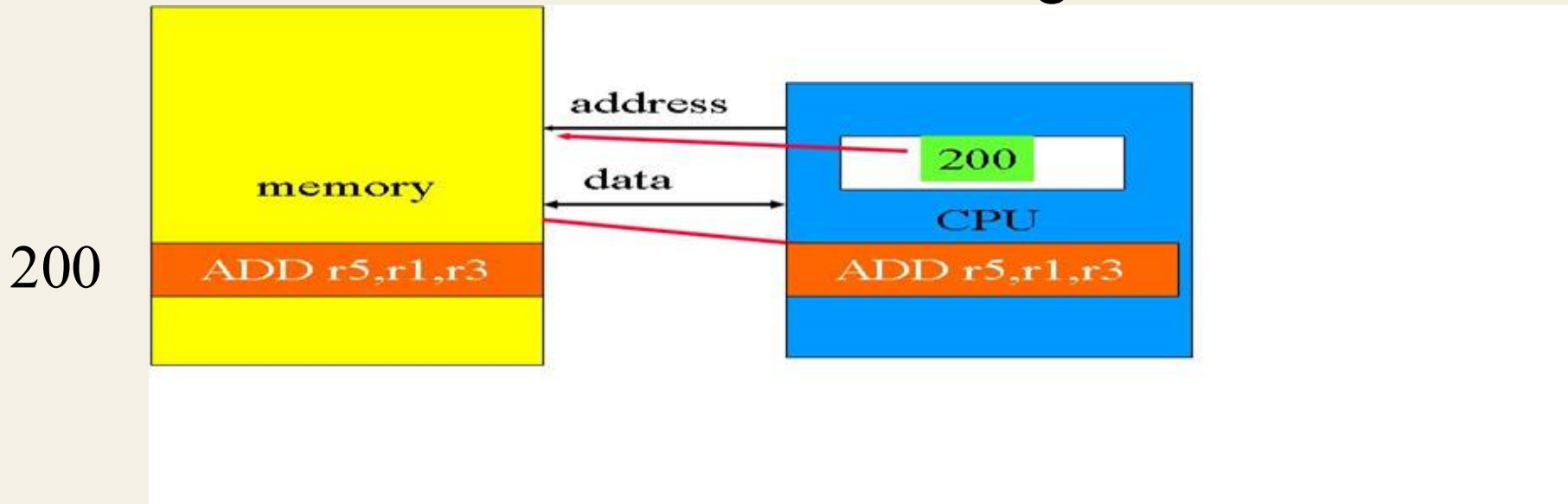
Embedded System HW Design
Week3
Comparing Processor Architectures

- **Type of Architectures**
  - Von Neumann
  - Harvard

- **Type of instruction sets**
  - **C**omplex **I**nstruction **S**et **C**omputers **(CISC)**
  - **R**educed **I**nstruction **S**et **C**omputers **(RISC)**

Embedded System HW Design
Week3
Von Neumann Architecture

- **Von Neumann Architecture**
  - Memory holds both data and instruction
  - Can be read or written when given an address



Embedded System HW Design
Week3
Harvard Architecture

- **Harvard Architecture**
  - Separate memory for data and program
  - Program counter (PC) points to program memory not data memory
  - Widely used today because provides higher performance

Embedded System HW Design
Week3
CISC and RISC

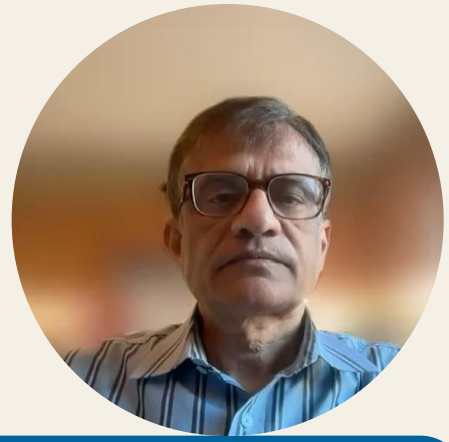
- **C**omplex **I**nstruction **S**et **C**omputers (CISC)
  - Instructions to provide complex tasks
  - x86

- **R**educed **I**nstruction **S**et **C**omputers (RISC)
  - Provides fewer and simpler instructions
  - Efficiently executed in pipelined processors
  - Also referred as **Load/Store Architecture**
  - ARM, RISC-V

- Most embedded today = ARM/RISC-V (RISC)

Embedded System HW Design
Week3
Comparing Processor Architectures

- *Choosing the right “brain” for an embedded system*



Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week3
Comparing Processor Architectures
Why Processor Architecture Matters

- Architecture impacts:
  - Performance
  - Power consumption
  - Cost
  - Complexity
  - OS support (bare-metal vs Linux)
- Wrong choice → redesign and schedule delays

Embedded System HW Design
Week3
Comparing Processor Architectures
Key Processor Categories Architecture

- MCU (Microcontroller)
- MPU (Microprocessor)
- SoC (System-on-Chip)
- DSP (Digital Signal Processor)
- DSC (Digital Signal Controller)

Template designed by Tim Palmer, tpalmer@vsu.edu

Embedded System HW Design
Week3
Comparing Processor Architectures
MCU Architecture

- Best for control + real-time systems
- CPU + Flash + RAM + peripherals on one chip
- Runs bare-metal or RTOS
- Low power, low cost, fast boot
- Examples: ARM Cortex-M, AVR, PIC

Template designed by Tim Palmer, tpalmer@ucsd.edu

Embedded System HW Design
Week3
Comparing Processor Architectures
MPU Architecture

- Best for high-level computing + Linux
- **CPU core usually requires:**
  - external DRAM
  - external Flash / storage
- Runs Linux or Android
- Higher performance, higher power, slower boot
- Examples: ARM Cortex-A, x86

Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week3
Comparing Processor Architectures
SoC Architecture

- MPU + major subsystems integrated
- CPU + GPU + accelerators + peripherals
- Multimedia + AI + high-speed I/O
- Often includes security blocks
- Examples: Snapdragon, Apple SoCs, NVIDIA Jetson, i.MX

Template designed by Tim Palmer, tpalmer@vsu.edu

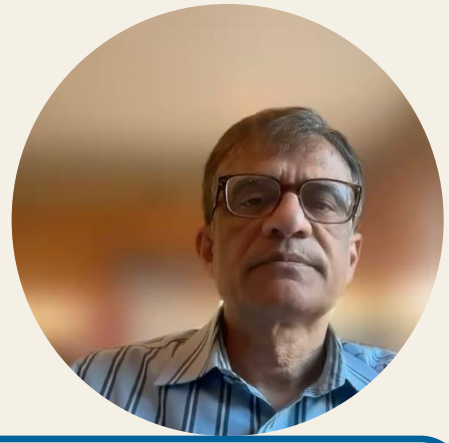
Embedded System HW Design
Week3
Comparing Processor Architectures
DSP Architecture

- Best for signal processing workloads
- **Optimized for:**
  - multiply-accumulate (MAC)
  - filters, FFT, audio, radar
- **Often used in:**
  - audio processing
  - communications
  - motor control (sometimes)
- Examples: TI C6000, ADI SHARC

Template designed by Tim Palmer, tpalmer@vsu.edu

Embedded System HW Design
Week3
Comparing Processor Architectures
DSC Architecture

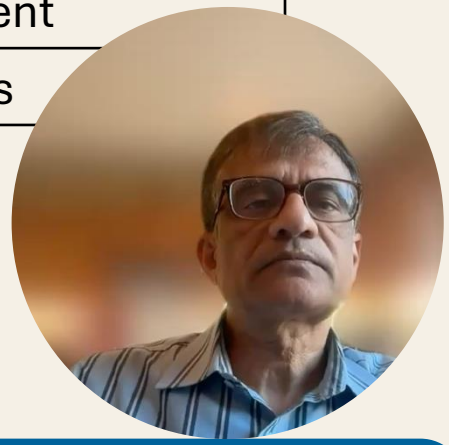
- MCU + DSP features
- **Combines:**
  - MCU control features
  - DSP math capability
- **Great for:**
  - motor control
  - power conversion
  - digital control loops
- Examples: Microchip dsPIC, TI C2000



Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week3
Comparing Processor Architectures
Architecture Comparison Table

Feature	MCU	MPU	SoC	DSP/DSC
Boot time	Fast	Slower	Slower	Fast
Power	Low	Higher	Higher	Medium
Cost	Low	Medium	Higher	Medium
OS	Bare/RTOS	Linux	Linux	Bare/RTOS
Real-time	Excellent	Harder	Harder	Excellent
Best for	Control	Computing	Multimedia/AI	Signals



Template designed by Tim Palmer, tpalmer@vsu.edu

Embedded System HW Design
Week3
Comparing Processor Architectures
How to Choose (Decision Guide)

Ask:

- Do I need **Linux**? → MPU/SoC
- Is it **real-time control**? → MCU/DSC
- Is it **heavy signal processing**? → DSP/DSC
- Is it **AI/multimedia**? → SoC
- What are **power + cost** constraints?

Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week3
Comparing Processor Architectures
Examples (Real Products)

- Smart thermostat → MCU
- Smart speaker / camera → SoC
- Router / gateway → MPU or SoC
- EV motor controller → DSC
- Audio noise cancellation → DSP

Template designed by Tim Palmer tpalmer@ucsd.edu

Embedded System HW Design
Week3
ARM Processors

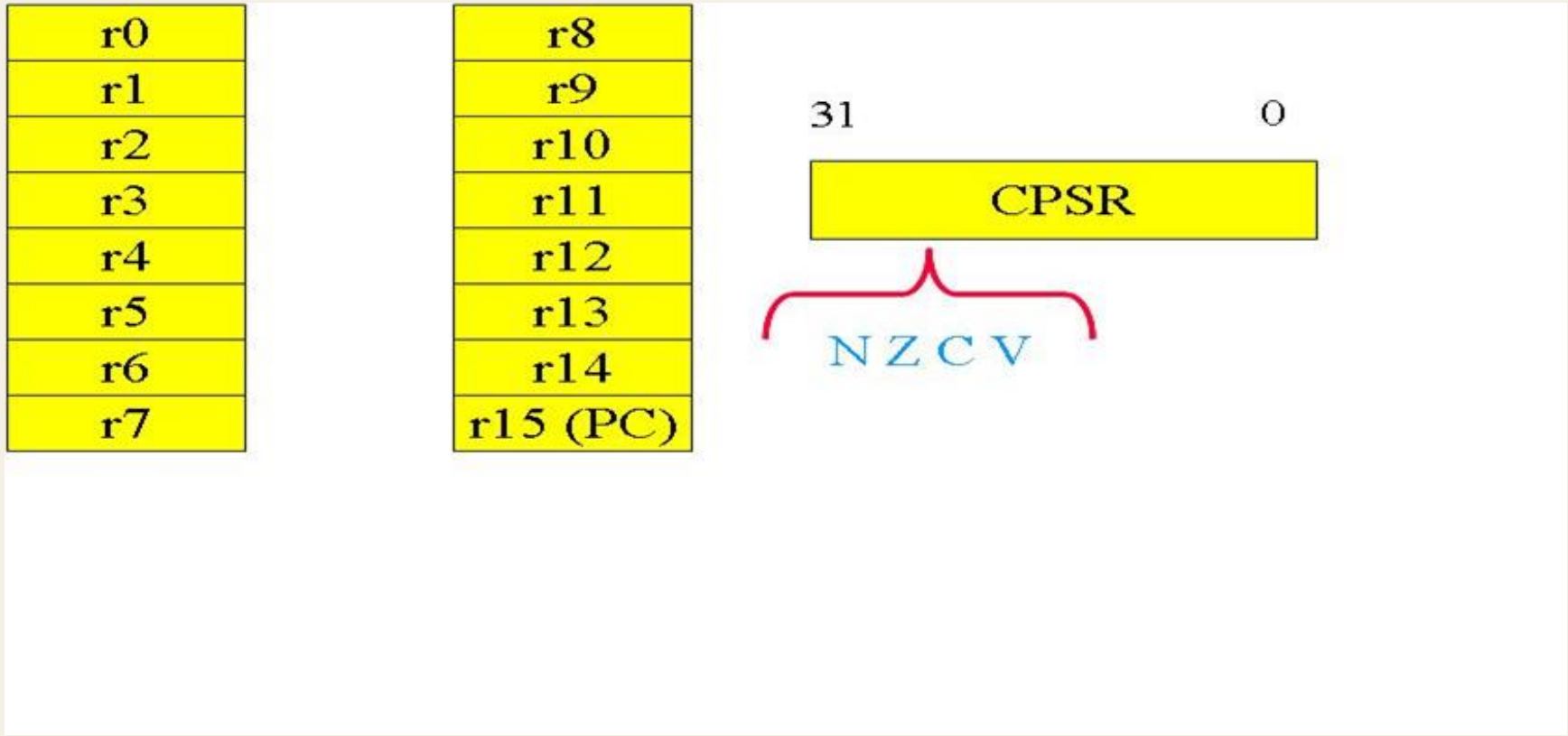
- **Topics Covered**
  - Processor and Memory organization
  - Data operation

Embedded System HW Design
Week3
ARM Processors
Instructions

- ARM processor uses RISC architecture
- ARM instructions are written one per line, starting after the first column
- Comments begin with a semi colon
- Label gives a name to a memory location, begins in the first column
- **Example:**

```
        LDR r0, [r8]        ; a comment
label   ADD r4, r0, r1
```

Week3
ARM Processors
Register Set

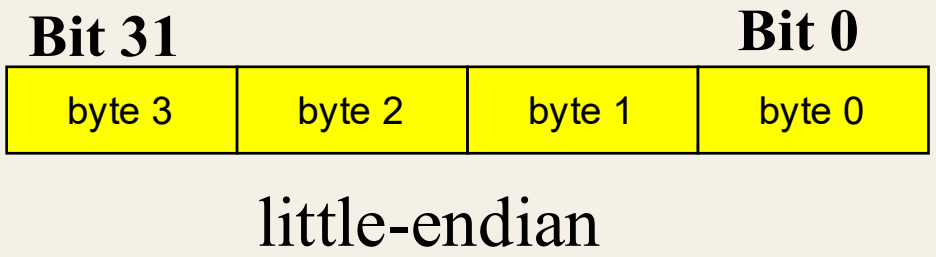


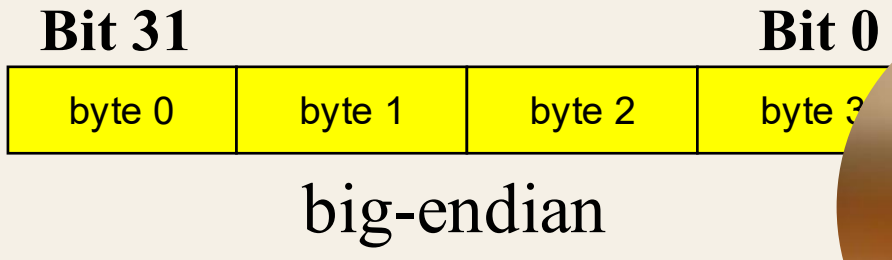
Embedded System HW Design
Week3
ARM Processors
Versions

- ARM architecture defined by different numbers
  - **ARM7** use Von Neumann architecture
  - **ARM9** use Harvard architecture

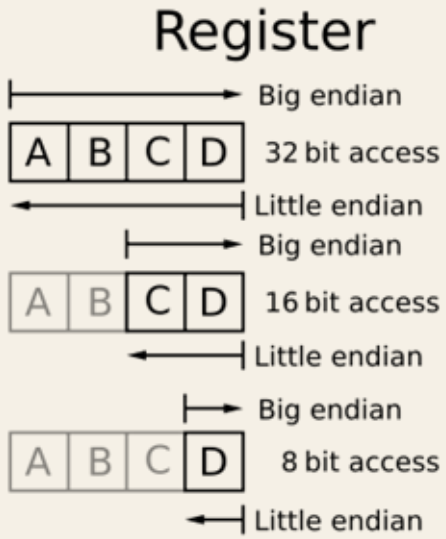
Embedded System HW Design
Week3
ARM Processors
Memory Organization

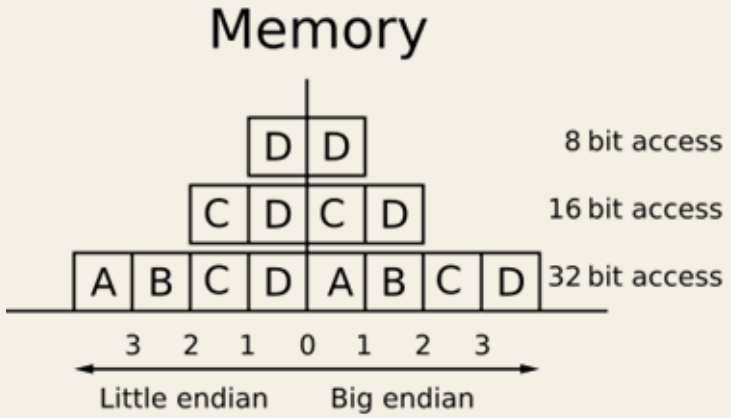
- ARM processor can be configured in two different addressing modes
  - **little-endian** mode
    - lowest-address byte stored in the least significant bits of the word
  - **big-endian** mode
    - lowest-address byte stored in the most significant bits of the word





Embedded System HW Design
Week3
ARM Processors
Memory Organization





Week3
ARM Processors
Data Operations

- Arithmetic and logical operations in C are performed in variables
- Variables are implemented as memory locations
- Arithmetic and logical operations cannot be performed directed on memory locations
- ARM uses **load-store architecture**- data operands are loaded first into the CPU from main memory, then stored back to main memory to save the results
- ARM has 16 general purpose registers **r0** to **r15**
- **r15** is used as **P**rogram **C**ounter
- **C**urrent **P**rogram **S**tatus **R**egister(CPSR) is set on every arithmetic, logical or shifting op
- CPSR holds NZCV status bits

Embedded System HW Design
Week3
ARM Processors
Status Bits

- **Every arithmetic, logical, or shifting operation sets CPSR bits:**

  N (negative), Z (zero), C (carry), V (overflow).

- **Example**
  - -1 + 1 = 0: NZCV=0110

Embedded System HW Design
Week3
ARM Processors
Data Instructions

Basic register operation

- ADD r0, r1, r2
  Computes r1 + r2 and stores result in r0

Immediate operation

- ADD r0, r1, #100
  Computes r1 + 100, and stores result in r0

Embedded System HW Design
Week3
ARM Processors
Data Instructions

- ADD, ADC : add (w. carry)
- SUB, SBC : subtract (w. carry)
- RSB, RSC : reverse subtract (w. carry)
- MUL, MLA : multiply (and accumulate)

AND : and operation

- BIC : bit clear
- LSL, LSR : logical shift left/right
- ASL, ASR : arithmetic shift left/right
- ROR : rotate right
- RRX : rotate right extended with C

Embedded System HW Design
Week3
ARM Processors
Comparison Instructions

- **CMP** : compare
- **CMN** : negated compare
- **TST** : bit-wise test
- **TEQ** : bit-wise negated test

These instructions set only the NZCV bits of CPSR.

Embedded System HW Design
Week3
ARM Processors
Move Instructions

- **MOV, (MVN)** : move (negated)

MOV r0, r1 ; sets r0 to r1

Week3
ARM Processors
Load/Store Instructions

- **LDR, LDRH, LDRB** : load (half-word, byte)

STR, STRH, STRB : store (half-word, byte)

Addressing modes:

Register indirect :

```
LDR r0,[r1]
```

With second register :

```
LDR r0,[r1,-r2]
```

With constant :

```
LDR r0,[r1,#4]
```

Embedded System HW Design
Week3
ARM Processors
Register Indirect Addressing

- ARM processor load and store instructions do not directly refer to main memory address

- Instead ARM uses **Register Indirect Addressing:**
  Value stored in the register is used as the address for the location from where the value is to be fetched

Example:
Location 0x100 is given a name **FOO**

```
ADR r1,FOO     ;Loads address(x0100) of FOO into r1

LDR r0, [r1]   ; Value of 0x100 is used as address.
               ; Contents at the address 100 is
               ; fetched and stored  in r0.
               ; If location 0x100 contained value of 0x55
               ; then r0 will be 0x55
```

Embedded System HW Design
Week3
ARM Processors
C assignments in ARM Instructions

Example

x = a + b;

```
ADR r4,a        ; get address of a
LDR r0,[r4]     ; get value of a
    ADR r4, b       ; get address of b
    LDR r1,[r4]     ; get value of b
    ADD r3, r0,r1 ; compute a + b
    ADR r4, x       ; get address of x
    STR r3, [r4]    ; store value of a+b into x
```

Embedded System HW Design
Week3
ARM Processors
Flow Control

- All operations can be performed conditionally, testing CPSR register:

EQ, NE, CS, CC, MI, GE, LT, GT, LE

- Branch operation:

B #100 ; unconditional branch

BEQ #100 ; Branch if previous operation result is equal

EQ	Equals; zero	Z= 1
NE	Not equl to zero	Z= 0
CS	Carry Set	C= 1
CC	Carry Clear	C= 0
MI	Minus	N= 0
GE	Great or equal	N = V
LT	Less than	N not
GT	Greater	
LE	Less or	

Embedded System HW Design
Week3
ARM Processors
Flow Control Example

C code:

```
if (a > b)
    {
      x = 5;
  y = c + d;
}
    else
      x = c - d;
```

Assembler:

```
        ; compute and test condition

        ADR r4,a  ; get address for a
        LDR r0,[r4]          ; get value of a
        ADR r4,b  ; get address for b
        LDR r1,[r4]          ; get value for b
        CMP r0,r1            ; compare a < b
        BGE fblock           ; if a >= b,branch to false block
```

Embedded System HW Design
Week3
ARM Processors
Flow Control Example Contd.

; true block of if (a > b)

```
MOV r0,#5          ; generate value for x
ADR r4,x           ; get address for x
STR r0,[r4]   ; store x
ADR r4,c           ; get address for c
LDR r0,[r4]   ; get value of c
ADR r4,d           ; get address for d
LDR r1,[r4]   ; get value of d
ADD r0,r0,r1 ; compute y
ADR r4,y           ; get address for y
STR r0,[r4]    ; store y
B after      ; branch around false block
```

Embedded System HW Design
Week3
ARM Processors
Flow Control Example Contd.

; false block of if (a > b)

```
fblock    ADR r4,c ; get address for c
          LDR r0,[r4] ; get value of c
          ADR r4,d ; get address for d
          LDR r1,[r4] ; get value for d
          SUB r0,r0,r1 ; compute a-b
          ADR r4,x ; get address for x
          STR r0,[r4] ; store value of x
```

after ...

Embedded System HW Design
Week3
ARM Processors
Flow Control Example Contd.

- For Loop

C implementation:

```
for ( i = 0; i < 4 ; i++)
x = x + i
```

Assembly:

```
        ; loop initialization
        MOV r0,#0          ; use r0 for i, set to 0
        MOV r1, #4         ; get value for loop termination test
        ADR r5, x          ; load r5 with address of x
loop    LDR r6, [x]        ; get value of x
        ADD r6, r6 + r0    ; perform x = x + I
      ADD r0, r0, #1       ; add 1 to i
      ;test for exit
      CMP r0, r1
        BLT     loop       ; if i < 4, continue loop
loopend..
```

Embedded System HW Design
Week3
ARM Cortex-M architecture
Where Cortex-M Fits

- MCU vs MPU vs SoC
- Cortex-M is used in **microcontrollers**
- Focus: low power, real-time control, fast interrupt response

Template designed by Tim Palmer, tpalmer@vsu.edu

Embedded System HW Design
Week3
ARM Cortex-M architecture
Cortex-M Families

- Cortex-M0/M0+ → low cost + low power
- Cortex-M3 → general purpose control
- Cortex-M4 → adds DSP + often FPU
- Cortex-M7 → higher performance
- Cortex-M33 → security + low power (TrustZone)

Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week3
ARM Cortex-M architecture
Cortex-M Core Building Blocks

- CPU core (executes instructions)
- NVIC (interrupt controller)
- SysTick timer
- Debug interface (SWD)
- Bus interface to memory/peripherals

Template designed by Tim Palmer, tpalmer@vsu.edu

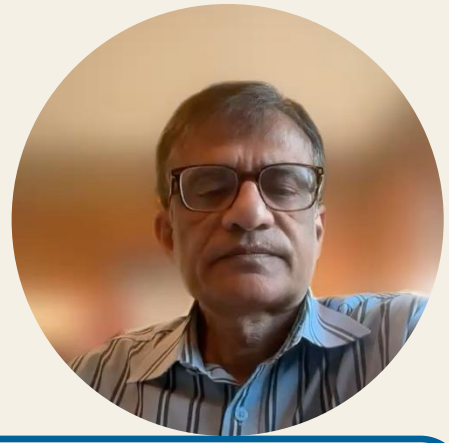
Embedded System HW Design
Week3
ARM Cortex-M architecture
Instruction Set Basics

- Thumb / Thumb-2 instruction set
- Optimized for code density (small firmware)
- Load/store architecture
- 32-bit registers

Template designed by Tim Palmer, tpalmer@vsu.edu

Embedded System HW Design
Week3
ARM Cortex-M architecture
Registers (What Firmware Uses)

- General purpose: R0–R12
- Stack pointer(s): MSP / PSP
- Link register (LR), Program counter (PC)
- Program status register (xPSR)



Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week3
ARM Cortex-M architecture
Exception & Interrupt Model

- Cortex-M is built for interrupts

- **Exceptions include:**
  - Reset
  - SysTick
  - peripheral interrupts
  - faults (HardFault, etc.)

- NVIC provides prioritization

Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week3
ARM Cortex-M architecture
NVIC (Nested Vectored Interrupt Controller)

- Handles interrupt vectors & priorities
- Nested interrupts (higher priority can preempt)
- Fast entry/exit = real-time advantage

Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week3
ARM Cortex-M architecture
Memory Map & Memory-Mapped I/O

- CPU sees everything via addresses

Address regions:

- Flash
- SRAM
- Peripheral registers
- Peripherals controlled by register reads/writes

Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week3
ARM Cortex-M architecture
Privilege, Protection & Faults

- Privileged vs unprivileged modes

- MPU (Memory Protection Unit) on some Cortex-M

Fault types:
- HardFault
- BusFault
- UsageFault
- Faults help debug serious problems

Template designed by Tim Palmer, tpalmer@ucsd.edu

Embedded System HW Design
Week3
ARM Cortex-M architecture
Debugging Features

- SWD/JTAG
- Breakpoints & watchpoints
- ITM / SWO trace (if supported)
- DWT cycle counter (timing measurements)

Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week3
ARM Cortex-M architecture
Cortex-M Summary (Key Takeaways)

Cortex-M optimized for:
- Low power
- Real-time interrupts
- Small embedded firmware

Core architectural pieces:
- NVIC + interrupts
- Memory mapped I/O

Template designed by Tim Palmer (palmer@ucsd.edu)

Embedded System HW Design
Week3
Links

Von Neumann:
http://en.wikipedia.org/wiki/Von_Neumann_architecture

Program Controlled Computers of 1940's
Colossus:
http://en.wikipedia.org/wiki/Colossus_computer

ENIAC:
http://en.wikipedia.org/wiki/ENIAC

Stored Program Computer:
http://en.wikipedia.org/wiki/EDVAC

Harvard:
http://en.wikipedia.org/wiki/Harvard_architecture

Endianness:
http://en.wikipedia.org/wiki/Endianness

Assembly Language:
http://en.wikipedia.org/wiki/Assembly_language

ARM:
http://http://en.wikipedia.org/wiki/ARM_architecture

Embedded System HW Design
Week3
Selecting Microcontrollers

- Selecting Microcontrollers
- Compare the different type of processor architectures
- ARM Cortex-M architecture

Template designed by Tim Palmer tpalmer@vsu.edu